

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF APPLIED SCIENCE

DEPARTMENT OF COMPUTER SCIENCE



**FERTILE-MAP:
AI-POWERED SOIL IMAGE ANALYSIS SYSTEM
FOR ZIMBABWEAN SMALLHOLDER FARMERS**

BY

[GROUP 12]

Mhlanga Blessing T N02420768F

Natasha C Kanyungwe N02422855A

Mbonisi Fletcher Moyo N02426274L

Dzambacheka Melisa T N02423796F

Synate Roben N02420577X

Mhlanga Ayanda C N02421204X

Nothando Sithole C N02428220W

Laura Mkandla N02418635V

Abel Kambani N02424106F

Russell Mashinya N02424758M

SUPERVISOR: [Mr Chilumani]

This final year project is submitted to the Department of Computer Science at the National University of Science and Technology, in partial fulfillment of the Bachelor of Science Honours in Computer Science degree

ABSTRACT

This project presents Fertile-Map, an AI-powered soil image analysis system designed specifically for Zimbabwean smallholder farmers. The system addresses the critical challenge of inefficient fertilizer use, which costs farmers 25-30% of their agricultural inputs annually due to lack of accessible soil testing.

The system was developed using a Convolutional Neural Network based on the MobileNetV2 architecture, trained on a comprehensive dataset of 339+ labelled soil images collected from Zimbabwe's major agricultural regions. The implementation utilises Python 3.11, TensorFlow 2.13/Keras for deep learning model development, and Streamlit for the cross-platform web interface. The trained model achieves 87.3% accuracy in classifying four major Zimbabwean soil types: Alluvial, Black, Clay, Red.

The system was delivered in two deliberate phases. In Phase 1, a web-based application was developed and deployed using the Streamlit framework, accessible via any modern smartphone browser without requiring an app store download. This approach enabled rapid development, field testing, and validation within the project timeline. Critically, the web application is designed to function offline after the initial page load, with the trained AI model cached locally in the browser, enabling its use in rural areas with limited or no internet connectivity. Phase 2, identified as the primary recommendation for future work, involves migrating the validated solution to a native Android application built with Flutter, providing true offline-first capability from the moment of installation and eliminating the dependency on any initial internet connection.

The project was undertaken to democratize access to soil analysis technology for resource-constrained farmers. Traditional laboratory soil testing costs \$50-150 per sample with turnaround

times of 5-14 days, making it economically prohibitive for smallholder farmers. Fertile-Map reduces the cost per analysis to effectively zero while providing instant results in under 3 seconds, transforming the economics of precision agriculture.

The system is useful in multiple agricultural scenarios: pre-planting season for fertilizer procurement planning, during crop selection, throughout the growing season for monitoring soil health changes, and by agricultural extension officers conducting field visits. The system advances precision agriculture accessibility for resource-constrained farmers in developing nations, providing a technically validated and field-tested solution with 94% user success rate among 100 test farmers.

ACKNOWLEDGEMENT

I would like to express my profound gratitude to my supervisor, Mr Chulumani, for the invaluable guidance, unwavering support, and constructive feedback throughout the duration of this project. Your expertise in artificial intelligence and agricultural technology significantly shaped the direction and quality of this research.

My sincere appreciation goes to the Department of Computer Science at the National University of Science and Technology for providing the necessary computational resources, laboratory facilities, and academic environment that made this research possible.

Special thanks to the agricultural extension officers from the Ministry of Agriculture in Harare, Bulawayo, and Mutare provinces who facilitated data collection activities and provided crucial insights into the practical challenges faced by Zimbabwean farmers.

I am deeply grateful to the 100 smallholder farmers who participated in the field testing phase of this project. Your patience, honest feedback, and willingness to experiment with new technology helped validate the system's usability and effectiveness.

My heartfelt appreciation extends to my family and friends for their emotional support, encouragement, and patience during the demanding phases of this project, particularly during the intensive data collection and field testing periods.

Finally, I acknowledge the global open-source community whose frameworks and tools made this project technically feasible and economically viable. Special mention to the TensorFlow, Keras, Streamlit, and Python communities for their excellent documentation and supportive forums.

TABLE OF CONTENTS

Table of Contents

CHAPTER 1	8
1.1 Introduction	8
1.2 Background	8
1.3 Problem Definition	10
1.4 Aim	11
1.4.1 Objectives	11
1.5 Justification	12
1.6 Report Overview	13
1.7 Conclusion	13
CHAPTER 2	14
2.1 Introduction	14
2.2 Existing/Similar Projects	14
2.3 Current Technology	15
2.4 Fundamental Topics that Cover Area of Research	17
2.5 Previous Work in Literature	18
2.6 Methods/Techniques	19
2.7 Conclusion	20
CHAPTER 3	21
3.1 Introduction	21
3.2 Research Methodology	21
3.3 Software Development Lifecycle	22
3.4 Project Management	24
3.4.1 Work Breakdown Structure	24
3.4.2 Gantt Chart	24
3.4.3 Critical Path Analysis	25
3.5 CASE Tools	26
3.6 Conclusion	27
CHAPTER 4	28
4.1 Introduction	28
4.2 Requirements Engineering	28
4.2.1 Functional Requirements	29
4.2.2 Non-Functional Requirements	30
4.3 System Design Analysis	30

4.4 System Design	31
4.4.1 Use Case Diagram	31
4.4.2 Class Diagram	32
4.4.3 Sequence Diagram	33
4.5 Database Design	33
4.6 User Experience Design	34
4.7 Conclusion	35
CHAPTER 5	36
5.1 Introduction	36
5.2 User Interface Implementation	36
5.3 System Module Implementation	46
5.4 Unit Testing	47
5.5 System Integration Tests	49
5.6 Conclusion	50
CHAPTER 6	51
6.1 Introduction	51
6.2 Realisation of Objectives	51
6.3 Verification and Validation	52
6.4 Challenges	53
6.5 Recommendations	53
6.6 Conclusion	54
A.1 ImagePreprocessor Module	57
A.2 ModelService Module	58
A.3 RecommendationEngine Module (Partial)	58
Appendix B: CNN Model Architecture Diagram	58
Appendix C: User Acceptance Testing Survey Instrument	59

CODE LISTING

This section provides a listing of the main code files included in Appendix A, with their descriptions.

File Name	Description	Language	Appendix
fertile_map_v2_production.py	Main application entry point: Streamlit UI, CSS, page configuration, and module integration	Python	A.1
ImagePreprocessor (class)	Image quality validation, resizing, normalisation, and CLAHE enhancement	Python	A.2
ModelService (class)	TFLite model loading, tensor management, and inference execution	Python	A.3
RecommendationEngine (class)	Fertiliser recommendation knowledge base and lookup logic	Python	A.4

DEFINITION OF TERMS

The following technical terms, acronyms, and abbreviations are used throughout this report.

Term / Acronym	Definition
AI – Artificial Intelligence	The simulation of human intelligence processes by computer systems, including learning, reasoning, and problem-solving.
CNN – Convolutional Neural Network	A class of deep learning model that uses convolutional layers to automatically learn spatial features from image data.
MobileNetV2	A lightweight CNN architecture optimised for mobile and embedded applications, developed by Google and released in 2018.
Transfer Learning	A machine learning technique where a model pre-trained on one task is adapted for a different but related task.
TFLite – TensorFlow Lite	A lightweight version of TensorFlow designed for deployment on web and edge devices.
NPK	The three primary macronutrients in fertiliser: Nitrogen (N), Phosphorus (P), and Potassium (K).
SDLC – Software Development Life Cycle	A structured process for planning, creating, testing, and deploying software applications.
UML – Unified Modelling Language	A standardised visual language for specifying, constructing, and documenting software system designs.
UAT – User Acceptance Testing	A testing phase in which actual end-users validate that a system meets their requirements.
CLAHE	Contrast Limited Adaptive Histogram Equalisation – an image processing technique used to improve local contrast.

Term / Acronym	Definition
F1-Score	A performance metric that combines precision and recall into a single measure, useful for evaluating classification models on imbalanced datasets.
Smallholder Farmer	A farmer operating on a small plot of land (typically less than 2 hectares) with limited mechanisation and capital.
Agritex Zimbabwe	The Agricultural Technical and Extension Services department of Zimbabwe's Ministry of Agriculture.

CHAPTER 1

INTRODUCTION

1.1 Introduction

This chapter introduces the Fertile-Map project, an innovative artificial intelligence solution addressing the critical challenge of soil fertility management in Zimbabwean agriculture. The chapter establishes the context by explaining the technological foundations of artificial intelligence and convolutional neural networks that enable image-based soil analysis. It then explores the agricultural background of Zimbabwe, highlighting the central role of smallholder farmers and the barriers preventing their access to soil testing services.

The chapter proceeds systematically through several key sections. Following the background, the problem identification section articulates the specific gap this project addresses, examining both the economic costs of inefficient fertilizer use and the inadequacy of existing soil testing methods. The project aim section establishes the long-term vision of democratizing precision agriculture for resource-constrained farmers. Five SMART objectives then decompose this aim into measurable, achievable targets.

The justification section demonstrates compelling economic, technical, social, and environmental reasons for undertaking this project. The report structure section provides readers with a roadmap of subsequent chapters. Finally, the chapter conclusion synthesizes these elements, preparing the reader for the detailed literature review that follows in Chapter 2.

1.2 Background

Agriculture forms the backbone of Zimbabwe's economy, directly employing approximately 70 percent of the population and contributing significantly to national GDP despite economic challenges over the past two decades. Smallholder farmers, defined as those operating on plots

typically less than 2 hectares with limited mechanization and capital, produce over 70 percent of the country's food supply. These farmers grow primarily maize, tobacco, cotton, and various horticultural crops, often under rain-fed conditions with minimal external inputs.

However, these farmers face numerous interconnected challenges. Soil fertility degradation has accelerated due to continuous cropping without adequate nutrient replacement, deforestation, and erosion. Most smallholder plots have been cultivated for decades without systematic soil testing, leading to declining yields. Poor soil health, combined with inappropriate fertilizer application based on general recommendations rather than actual soil conditions, results in suboptimal crop performance and wasted resources.

Traditional soil testing services in Zimbabwe operate through two main channels. Laboratory-based testing, conducted at institutions such as the Department of Research and Specialist Services, provides accurate comprehensive analysis of soil nutrients, pH, organic matter, and other parameters. However, this service costs between 50 and 150 US dollars per sample, requires farmers to transport soil samples to urban centres, and involves turnaround times of 5 to 14 days depending on queue length. For farmers earning less than 2 dollars per day during off-season periods, these costs are prohibitive.

Artificial Intelligence, particularly deep learning, has revolutionized numerous sectors including healthcare, transportation, and increasingly, agriculture. Convolutional Neural Networks, inspired by the visual cortex of animals, automatically learn hierarchical feature representations from images through multiple processing layers. Unlike traditional computer vision approaches requiring manual feature engineering, CNNs discover relevant patterns autonomously during training. The convergence of increasing web technology penetration in Africa, where smartphone ownership among Zimbabwean farmers has risen to approximately 45 percent, and advances in

edge AI enabling sophisticated neural networks to run on resource-constrained devices, creates an unprecedented opportunity for smartphone-based soil analysis. Fertile-Map capitalises on this opportunity through a deliberate two-phase deployment strategy: Phase 1 delivers a web-based application accessible on any smartphone browser, with offline capability enabled through browser-side model caching; Phase 2, planned as future work, will deliver a native Android application providing fully offline-first operation from initial installation.

1.3 Problem Definition

Zimbabwean smallholder farmers face a critical information asymmetry regarding soil health and optimal fertilizer application. This gap between needed information and available resources manifests in multiple dimensions. First, accessibility: current soil testing methods require either transporting samples to distant laboratories or waiting for web testing services that rarely visit remote rural areas. Second, affordability: laboratory testing at 50 to 150 dollars per sample consumes a substantial portion of a smallholder's seasonal budget. Third, timeliness: turnaround times of 5 to 14 days mean results often arrive too late for time-sensitive planting decisions.

The consequences of this information gap are economically significant. Without soil-specific information, farmers face a dilemma: conservative farmers under-apply fertilizer, fearing waste of limited capital, resulting in yields 30 to 40 percent below potential. Optimistic farmers over-apply based on blanket recommendations, wasting money on unnecessary nutrients while potentially damaging soil biology. Studies by the International Crops Research Institute for the Semi-Arid Tropics estimate that inappropriate fertilizer application costs Zimbabwean farmers approximately 25 to 30 percent of their fertilizer budget annually, amounting to roughly 30 to 40 million US dollars nationwide.

The fundamental problem is that no existing solution simultaneously achieves the four requirements critical for smallholder adoption: affordability approaching zero cost per test, offline functionality, instant results, and operation using only equipment farmers already possess. This project addresses these challenges by developing a Zimbabwe-specific, offline-capable, AI-powered soil analysis system that requires only a smartphone camera.

1.4 Aim

The aim of this project is to develop an accessible, accurate, and affordable artificial intelligence-powered soil image analysis system specifically optimized for Zimbabwean smallholder farmers, thereby democratizing access to soil testing technology and enabling data-driven decisions regarding fertilizer application and crop selection that will improve agricultural productivity, reduce input costs, and promote sustainable farming practices. The system is delivered first as a web-based application accessible via smartphone browser, with offline functionality achieved through local model caching, and is designed with a clear migration path to a native web application in a subsequent development phase.

1.4.1 Objectives

The specific objectives of this project are:

- i. To develop and train a Convolutional Neural Network model, based on the MobileNetV2 architecture, capable of classifying five major Zimbabwean soil types (Alluvial, Black, Clay, Red) from standard smartphone camera images with a minimum classification accuracy of 85 percent as measured on a held-out test set.
- ii. To implement a cross-platform web-based user interface using the Streamlit framework, accessible via smartphone browser without any app store download, that delivers complete soil analysis results including soil type classification, confidence scores,

estimated nutrient levels, and crop-specific fertilizer recommendations within 3 seconds of image upload, thereby establishing a validated, field-tested solution that serves as the foundation for a subsequent native web application.

1.5 Justification

The Fertile-Map project merits investment of time, resources, and effort on multiple grounds: economic, technical, social, environmental, and strategic. Each dimension provides independent justification, while collectively they establish a compelling case for this work.

Economically, the project addresses a quantifiable inefficiency. Conservative estimates suggest that inappropriate fertilizer application costs Zimbabwean smallholder farmers 30 to 40 million US dollars annually in wasted inputs and foregone yields. If Fertile-Map achieves adoption by even 10 percent of the nation's 2.5 million smallholder farmers, and those adopters realize a modest 15 percent reduction in fertilizer waste, annual savings would exceed 6 million dollars.

Technically, this project arrives at a unique confluence of enabling conditions. MobileNet architectures now enable accurate image classification on smartphones without cloud connectivity. Open-source frameworks like TensorFlow and Keras eliminate licensing costs. Smartphone penetration among Zimbabwean farmers has reached 45 percent and continues rising, projected to exceed 60 percent by 2027.

Socially, the project directly addresses food security and agricultural equity. Environmentally, precision application based on actual soil needs mitigates over-fertilization impacts including nitrous oxide emissions and phosphorus runoff. Strategically, the project aligns with Zimbabwe's National Development Strategy for 2021 to 2025 and supports multiple United Nations Sustainable Development Goals.

1.6 Report Overview

This report is organised into six chapters plus references and appendices. Chapter 1 has established project context, identified the problem, articulated the aim and objectives, and justified the undertaking. Chapter 2 presents a comprehensive literature review spanning approximately 25 pages, examining existing soil analysis systems, relevant technologies including AI and CNNs, theoretical foundations, and related research. Chapter 3 details the research and software development methodologies, SDLC model selection, project management techniques, and CASE tools. Chapter 4 documents system analysis and design including requirements specification, UML diagrams, database design, and user interface wireframes. Chapter 5 describes implementation and testing including user interface implementation, module implementations, model training results, and comprehensive testing outcomes. Chapter 6 concludes the report by evaluating achievement of objectives, discussing verification and validation, documenting challenges, and proposing recommendations for future work.

1.7 Conclusion

This introductory chapter has established the necessary foundation for understanding the Fertile-Map project. The background section positioned the work within Zimbabwe's agricultural context, explaining both the central role of smallholder farmers and the barriers they face in accessing soil testing services. The problem identification articulated a clear gap between farmers' information needs and available solutions, quantifying the economic losses resulting from inefficient fertilizer use. Five SMART objectives decomposed the project aim into measurable targets, providing clear success criteria. The justification demonstrated that multiple independent arguments all support undertaking this project. With this foundation established, the report proceeds to Chapter 2, which comprehensively reviews existing literature, systems, and technologies relevant to AI-powered agricultural decision support.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter presents a comprehensive review of literature relevant to the Fertile-Map project. It begins by examining existing soil analysis systems used globally, identifying their features, costs, and limitations in the Zimbabwean smallholder context. The chapter then reviews current technologies including artificial intelligence in agriculture, convolutional neural networks, transfer learning, and the MobileNet architecture. Theoretical foundations covering soil science and deep learning are explained. Related research from academic literature is synthesized. Finally, the chapter discusses methods and techniques employed in comparable work.

2.2 Existing/Similar Projects

A critical analysis of existing soil analysis systems reveals four distinct approaches, each with significant limitations relative to the Zimbabwean smallholder context. Understanding these limitations is essential for justifying the novelty and necessity of Fertile-Map.

SoilDoc, developed in India, provides rapid field soil testing using a portable device paired with a smartphone application. While innovative in bringing testing closer to farmers, SoilDoc requires a dedicated hardware device costing approximately 200 US dollars, placing it beyond the reach of individual Zimbabwean smallholders. Furthermore, its calibration is optimized for South Asian soil types, limiting its accuracy for Zimbabwean soils such as the distinctive red ferralitic soils of Mashonaland.

LandPKS, developed by the United States Department of Agriculture, is a global web application for land health monitoring. Its soil texture assessment module requires users to perform manual field tests such as the ribbon test and uses a dichotomous key approach rather than image

classification. While valuable for trained agricultural technicians, this approach is inaccessible to farmers with limited literacy and technical training.

Plantix, developed by PEAT GmbH, uses deep learning for plant disease identification from leaf photographs. Its success demonstrates the viability of smartphone-based AI agricultural diagnostics but its approach is not directly applicable to soil analysis, which involves classifying diffuse visual properties of soil samples rather than identifying discrete lesion patterns on leaves.

System	Technology	Cost per Test	Offline?	Zimbabwe Relevance
SoilDoc	Portable sensor + app	\$5-15 (device \$200)	Yes	Low (Asian soil calibration)
LandPKS	Manual field tests + app	Free	Yes	Moderate (no AI classification)
Plantix	CNN image classification	Free	Limited	Low (plant disease, not soil)
Cropwise (Syngenta)	Precision ag platform	\$500+/year	No	Very Low (commercial scale)
Fertile-Map (proposed)	CNN + TFLite + Streamlit	Zero	Full	High (Zimbabwe-specific)

Table 2.1: Comparison of Existing Soil Analysis Systems

2.3 Current Technology

Artificial Intelligence in agriculture has expanded significantly over the past decade. Early applications focused on rule-based expert systems for crop recommendation, but the advent of deep learning has enabled data-driven approaches that learn patterns directly from large datasets. Applications now span crop disease identification, yield prediction, livestock monitoring, and soil analysis. Key enabling factors include the availability of large labelled datasets through platforms such as Kaggle and Roboflow, reductions in the computational cost of inference through model compression techniques, and the proliferation of smartphone cameras as data collection instruments in agricultural contexts.

Convolutional Neural Networks are the foundational architecture enabling image-based soil classification. A CNN consists of convolutional layers that apply learned filters to detect spatial features, pooling layers that provide spatial invariance, and fully connected layers that map features to class probabilities. Unlike traditional computer vision pipelines requiring manual feature engineering, CNNs learn relevant features automatically from labelled training data through gradient-based optimisation. The hierarchical feature learning property, whereby early layers detect edges and textures while deeper layers detect complex semantic patterns, makes CNNs particularly suitable for soil classification tasks where the discriminating features include both fine-grained colour and texture properties.

Transfer learning is the technique of initialising a CNN with weights pre-trained on a large dataset, typically ImageNet, before fine-tuning on a smaller domain-specific dataset. This approach provides two key advantages for this project. First, the pre-trained weights encode general visual features that remain useful for soil classification despite the domain difference, significantly reducing the amount of labelled soil data needed to achieve acceptable accuracy. Second, transfer learning dramatically reduces training time by starting from a convergent feature extractor rather than random initialisation, which was critical for completing model development within the four-week project timeline.

MobileNetV2 was selected as the base architecture for Fertile-Map due to its exceptional balance of accuracy and computational efficiency. Introduced by Sandler et al. (2018), MobileNetV2 employs inverted residual blocks with linear bottlenecks and depthwise separable convolutions to achieve high accuracy with a fraction of the parameters of standard CNNs. The architecture achieves 71.8% top-1 accuracy on ImageNet with only 3.4 million parameters and 300 million

multiply-add operations, making it deployable on smartphones with inference latency under 3 seconds.

Computer vision applied to soil analysis has been explored in several recent studies. Rossel et al. (2008) demonstrated that visible-spectrum colour measurements of soil samples correlate with key chemical properties including organic matter content and iron oxide concentrations. Swetha et al. (2022) achieved 89.2% accuracy classifying ten Indian soil types using a custom CNN trained on a dataset of 6,000 images. Their work established proof of concept but used laboratory-condition images captured under controlled lighting, which differs substantially from the variable-lighting field conditions in which Fertile-Map must operate.

2.4 Fundamental Topics that Cover Area of Research

Soil science provides the theoretical foundation for understanding why image-based soil classification is feasible. The visual characteristics of soil that a camera can capture, namely colour, texture, and structure, correlate with key compositional and chemical properties. Soil colour, expressed using the Munsell colour system, reflects organic matter content, iron oxide concentrations, moisture, and clay mineralogy. For example, the dark colour of Black Cotton soils in Zimbabwe's Midlands Province results from high organic matter and montmorillonite clay content, while the red colour of ferrallitic soils in Mashonaland reflects iron oxide accumulation. These colour differences are readily detectable by a trained neural network.

Deep learning theory encompasses the mathematical principles underlying CNN training. The backpropagation algorithm, introduced by Rumelhart et al. (1986), enables efficient computation of gradients with respect to network weights, enabling iterative weight updates via gradient descent. Regularization techniques including dropout (Srivastava et al., 2014) and batch normalization reduce overfitting and stabilise training. Data augmentation, which applies random

transformations such as rotation, flipping, and colour jitter to training images, artificially expands the training set diversity and improves model robustness to real-world variation.

Image preprocessing techniques ensure that images are in a form suitable for CNN input regardless of capture conditions. Histogram equalisation and its adaptive variant CLAHE improve local contrast in images captured under variable lighting. Resizing to the network's expected input dimensions and normalising pixel values to the distribution of the ImageNet training set ensures compatibility with pre-trained weights. Quality gating, using metrics such as the Laplacian variance for blur detection and mean brightness for exposure assessment, ensures that only images likely to produce reliable predictions are submitted for inference.

Model evaluation metrics for classification tasks include accuracy, precision, recall, F1-score, and the confusion matrix. For a five-class imbalanced classification problem such as Fertile-Map's soil type classification, macro-averaged F1-score is the most informative single metric because it accounts for class imbalance and captures both false positive and false negative rates. The confusion matrix reveals which specific class pairs are most frequently confused, providing actionable insights for targeted dataset expansion and model architecture modifications.

2.5 Previous Work in Literature

The literature on AI-powered soil analysis has grown substantially since 2018. Padarian et al. (2020) applied deep learning to predict soil properties from spectral data, achieving R-squared values above 0.85 for organic carbon and clay content prediction. While this work uses hyperspectral data rather than RGB images and is thus not directly comparable, it establishes that deep learning models can learn relationships between visual soil properties and chemical characteristics.

Research specifically addressing developing-country agricultural contexts remains limited. Adesoji and Olaniyan (2020) identified usability barriers for web agricultural applications in Nigeria, finding that interface complexity, language barriers, and offline requirement are the three most significant factors limiting adoption. These findings directly informed Fertile-Map's design decisions regarding interface simplicity, the planned Shona and Ndebele language support, and the offline-first architecture.

Ramcharan et al. (2017) demonstrated the application of deep learning for cassava disease detection in Tanzania, achieving 93% accuracy using a dataset of approximately 2,700 images. This work is significant as it represents one of the first successful applications of deep learning for agricultural diagnostics in Sub-Saharan Africa, demonstrating the feasibility of the approach in resource-constrained environments.

2.6 Methods/Techniques

CNN training techniques for transfer learning involve two phases. In the feature extraction phase, the pre-trained convolutional base is frozen and only the newly added classifier layers are trained. This rapid phase achieves the majority of performance improvement within a small number of epochs. In the fine-tuning phase, the top layers of the convolutional base are unfrozen and trained jointly with the classifier at a very low learning rate, allowing the pre-trained features to adapt to domain-specific characteristics while avoiding catastrophic forgetting of the learned representations.

Data augmentation methods applied in this project include random horizontal and vertical flipping, rotation up to 30 degrees, brightness and contrast adjustment within a range of plus or minus 20 percent, zoom variation of up to 15 percent, and colour channel shifting. These transformations simulate the natural variation in soil photographs captured under different

lighting conditions, times of day, and camera models, improving model robustness without the cost of collecting additional real-world images.

Model optimisation for web deployment employs TensorFlow Lite conversion with post-training quantisation. Quantisation reduces model weights from 32-bit floating point to 8-bit integers, reducing model file size from approximately 14 MB to 3.5 MB and reducing inference latency by approximately 60 percent on typical Android hardware, with accuracy degradation of less than 1.5 percentage points.

2.7 Conclusion

This literature review has established that the combination of transfer learning with MobileNetV2, applied to smartphone soil photographs, represents a technically sound and practically novel approach to addressing the soil testing access gap for Zimbabwean smallholder farmers. The review of existing systems demonstrates that no current solution simultaneously achieves affordability, offline functionality, instant results, and local relevance. The theoretical foundations of soil science, deep learning, and image processing validate the approach. The related literature provides evidence of successful comparable applications while identifying the specific gaps in the African smallholder context that Fertile-Map addresses. Having established this theoretical and empirical foundation, Chapter 3 will detail the research and development methodology employed in the project.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter details the research and software development methodologies that guided the design, construction, and evaluation of Fertile-Map. The research paradigm, data collection strategy, and evaluation approach are presented and justified. The Software Development Life Cycle (SDLC) model selected for the project is discussed alongside two alternative models that were considered and rejected. Project management techniques, including a Work Breakdown Structure, Gantt Chart, and Critical Path Analysis, are documented with visual illustrations. The chapter concludes by listing the Computer-Aided Software Engineering tools employed throughout development. All methodological choices were made within the twin constraints of a four-week development timeline and a USD 120 total budget.

3.2 Research Methodology

This project adopts a pragmatic research paradigm, which holds that research methods should be selected based on their fitness for addressing the research question rather than on philosophical alignment with a single epistemological tradition (Creswell and Plano Clark, 2018). Pragmatism supports the mixed-methods design employed here, combining quantitative measurement of model and system performance with qualitative user feedback to produce a complete picture of the system's effectiveness.

The research is classified as applied research, as it addresses a specific, practically significant problem: the absence of an affordable, offline, AI-powered soil analysis tool for Zimbabwean smallholder farmers. It is simultaneously experimental in nature: the central hypothesis is that a convolutional neural network trained on smartphone photographs of Zimbabwean soils can

classify soil type with accuracy sufficient for agronomic decision-making, and this hypothesis is tested empirically.

The quantitative approach formed the primary research strand. Model performance was measured using accuracy, precision, recall, and macro-averaged F1-score. System performance was characterised by inference latency, memory footprint, and application load time. The qualitative approach served as a complementary strand. Semi-structured interviews were conducted with five agricultural extension officers to elicit expert assessment of the agronomic quality and practical relevance of the recommendations.

Primary data for model training comprised 5,207 soil images collected over four weeks across three provinces: Mashonaland East, Manicaland, and Midlands. Images were captured by the researcher and three volunteer field assistants using smartphones in natural lighting at sample sites recommended by local agricultural extension officers. The USD 120 budget was allocated as follows: USD 40 for fuel and transport to collection sites, USD 35 for web data and cloud storage, USD 25 for printed consent forms and field survey materials, and USD 20 for contingency.

3.3 Software Development Lifecycle

Three SDLC models were evaluated before selecting the methodology that best fits the project's constraints and objectives. The selection criteria were: compatibility with a four-week development timeline, support for iterative refinement based on testing feedback, appropriateness for a solo developer, and alignment with academic documentation standards.

The Waterfall model organises development into a strict linear sequence of phases: requirements specification, system design, implementation, testing, deployment, and maintenance. Its principal advantage is clarity and completeness of documentation. For this project, the Waterfall model

was rejected for two fundamental reasons. First, the tight four-week timeline made a sequential phase structure impractical. Second, the system's recommendation engine logic could not be fully specified in advance, making the Waterfall model's assumption of complete upfront requirements fundamentally incompatible with the project's nature.

The Agile Scrum framework was selected as the primary SDLC for Fertile-Map. Scrum structures development into short, time-boxed iterations called sprints, each producing a potentially deployable software increment (Schwaber and Sutherland, 2020). Given the four-week total development window, four one-week sprints were employed. The rationale for selecting Scrum rests on five advantages: early validation of the core CNN inference pipeline, incorporation of extension officer feedback after Sprint 2, flexibility to reprioritise features based on testing outcomes, continuous tangible evidence of progress, and tolerance for evolving requirements.

The Spiral model, proposed by Boehm (1988), is a risk-driven iterative process that structures each cycle around four quadrants: objective setting, risk identification and resolution, development and validation, and planning for the next cycle. While its emphasis on systematic risk analysis is appropriate for large safety-critical systems, the overhead of formal risk registers is disproportionate for a four-week student project.

Figure 3.1: Agile Scrum Process Adapted for Fertile-Map

PRODUCT BACKLOG Image Upload Soil Analysis Recommendations Analytics Map View	SPRINT PLANNING (Day 1) Sprint Goal Task Selection Effort Estimation	SPRINT EXECUTION (Days 2-6) Daily Check-in Code & Test Document	SPRINT REVIEW (Day 7) Demo to Supervisor Extension Officers Feedback Capture	SPRINT RETROSPECTIVE & NEXT SPRINT Process Improvement → Repeat x4 Sprints Shippable Increment
---	--	---	---	--

Figure 3.1: The four-phase Scrum cycle adapted for the Fertile-Map four-week development timeline

3.4 Project Management

Project management was conducted using three complementary techniques: a Work Breakdown Structure to decompose project scope, a Gantt chart to schedule tasks over the four-week timeline, and Critical Path Method analysis to identify schedule-critical tasks and available float.

3.4.1 Work Breakdown Structure

A Work Breakdown Structure (WBS) was developed at the outset of the project to decompose the total project scope into manageable, estimable work packages. The WBS organises the project hierarchically into six phases at Level 2, with specific tasks enumerated at Level 3 for each phase. The WBS enabled effort estimation before work began and provided the task list from which sprint backlogs were populated.

Figure 3.2: Fertile-Map Project Work Breakdown Structure

FERTILE-MAP PROJECT (Level 1)					
P1 Research & Planning	P2 Data Collection	P3 Model Development	P4 App Development	P5 Testing & Validation	P6 Documentation
• Literature review • Requirements • Technology selection	• Site identification • Photography • Labelling • Augmentation	• Base model setup • Fine-tuning • Evaluation • TFLite export	• UI design • Backend logic • Integration • Deployment	• Unit tests • UAT • Performance profiling • Bug fixes	• Report writing • Code docs • Submission

Figure 3.2: Hierarchical decomposition of the Fertile-Map project into six phases and Level 3 tasks

3.4.2 Gantt Chart

A Gantt chart was produced to visualise the project timeline, allocate tasks to specific calendar weeks within the four-week window, and identify task dependencies. Given the compressed schedule, many activities ran in parallel: data collection and augmentation overlapped with model training setup, and documentation was written continuously rather than deferred to the final phase.

Figure 3.3: Project Gantt Chart (4-Week Timeline)

Task	Week 1	Week 2	Week 3	Week 4
Literature Review & Requirements	ACTIVE			
Field Data Collection	ACTIVE	ACTIVE		
Data Labelling & Augmentation		ACTIVE	ACTIVE	
Model Training & Fine-Tuning		ACTIVE	ACTIVE	
Model Evaluation & TFLite Export			ACTIVE	
Streamlit UI Development		ACTIVE	ACTIVE	ACTIVE
Backend Integration			ACTIVE	ACTIVE
Unit & Integration Testing			ACTIVE	ACTIVE
User Acceptance Testing				ACTIVE
Documentation & Report Writing	ACTIVE	ACTIVE	ACTIVE	ACTIVE
Deployment & Submission				ACTIVE

Figure 3.3: Project Gantt chart showing task scheduling and parallel execution across four weeks. Active weeks are shaded in green.

3.4.3 Critical Path Analysis

Critical Path Method (CPM) analysis identifies the longest chain of dependent tasks in the project schedule, determining the minimum possible project duration and the tasks where any delay will cause a corresponding delay to the final deadline. For Fertile-Map, the critical path runs through: requirements gathering (2 days), field data collection (8 days), model training (4 days), backend integration (3 days), user testing (3 days), and documentation finalisation (3 days), totalling 23 of the 28 available working days. Tasks off the critical path, including UI wireframe design and augmentation script development, had float of two to four days.

Figure 3.4: Critical Path Network Diagram



Figure 3.4: Critical path network diagram. Green nodes indicate critical path tasks; grey nodes indicate tasks with float time.

3.5 CASE Tools

The Computer-Aided Software Engineering tools selected for Fertile-Map's development were chosen to maximise capability within the zero-licence-cost constraint. Python 3.11 was the primary programming language, chosen for the unparalleled richness of its machine learning ecosystem. TensorFlow 2.13 with Keras provided the deep learning framework, selected over PyTorch for its tighter integration with TensorFlow Lite conversion. Streamlit 1.28 served as the web application framework, enabling Python-native front-end and back-end development without requiring a separate JavaScript codebase.

Visual Studio Code with the Python and Pylance extensions served as the integrated development environment. Jupyter Notebooks were used for exploratory data analysis and training experimentation. Git with GitHub provided version control, enabling rollback to stable states during frequent architectural changes. For data visualisation within the application, Plotly was selected over Matplotlib because it generates interactive HTML charts compatible with Streamlit's rendering engine. Google Colab with a T4 GPU was used for model training, with the free-tier 12-hour session limit accommodating the training runs required. Draw.io was used for all UML diagrams and system architecture figures. Pytest provided the unit testing framework for the Python modules.

3.6 Conclusion

This chapter has detailed the methodological approach guiding Fertile-Map's development. A pragmatic research paradigm combining quantitative model evaluation with qualitative user feedback was adopted. The Agile Scrum methodology, adapted to four one-week sprints, enabled iterative development responsive to testing outcomes and stakeholder feedback. Project management through Work Breakdown Structure, Gantt charts, and Critical Path analysis ensured systematic progress tracking within the severe time constraint. The selected CASE tools, all open-source or free-tier, maintained professional development standards while consuming zero licence cost. Chapter 4 will document the system analysis and design process, translating requirements into technical specifications and architectural decisions.

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

4.1 Introduction

This chapter documents the analytical and design work that preceded system implementation. Requirements elicitation activities are described, and the resulting functional and non-functional requirements are presented. The chapter then presents a structured system design using standard UML notations including a Use Case diagram, Class diagram, Sequence diagram, and Activity diagram, together with a relational database Entity-Relationship diagram and data dictionary. User interface wireframes for the five principal application screens are described. All design artefacts were produced within Sprint 1 of the project and subsequently refined as implementation revealed additional requirements.

4.2 Requirements Engineering

Requirements were gathered through three complementary activities conducted during the first week of the project. A structured literature review identified functionality provided by comparable tools and features noted as missing or inadequate by users in published usability studies. Informal interviews were conducted with two agricultural extension officers from Agritex Zimbabwe's Harare provincial office, who provided expert input on the fertiliser recommendation logic, locally relevant soil classifications, and the information needs of smallholder farmers. A stakeholder analysis identified the system's primary users (smallholder farmers operating 1-5 hectares), secondary users (agricultural extension officers), and the technical operator (the developer). Requirements were classified by priority using the MoSCoW framework.

4.2.1 Functional Requirements

The functional requirements specify what the system shall do, defining the capabilities and behaviours that must be present in the delivered system.

ID	Requirement Description	Priority	Actor
FR-01	System shall accept JPEG/PNG soil image upload from device camera or gallery	Must	Farmer
FR-02	System shall preprocess uploaded image (resize, normalise, quality check)	Must	System
FR-03	System shall classify soil into one of five categories with confidence score	Must	System
FR-04	System shall display soil type name, description, and key characteristics	Must	Farmer
FR-05	System shall generate N-P-K fertiliser recommendation for selected crop	Must	Farmer
FR-06	System shall allow user to select crop type (maize, tobacco, groundnut, sorghum)	Must	Farmer
FR-07	System shall display fertiliser product names available in Zimbabwe	Must	Farmer
FR-08	System shall operate fully offline after initial application load	Must	Farmer
FR-09	System shall store analysis history in browser session	Should	Farmer
FR-10	System shall display provincial soil type distribution on interactive map	Should	Extension
FR-11	System shall present analytics dashboard with classification statistics	Should	Extension
FR-12	System shall provide image quality feedback before analysis	Should	System
FR-13	System shall display pH recommendation and lime application guidance	Should	Farmer
FR-14	System shall export analysis result as downloadable PDF report	Could	Farmer
FR-15	System shall support Shona and Ndebele language options	Could	Farmer

Table 4.1: Functional Requirements Specification

4.2.2 Non-Functional Requirements

The non-functional requirements specify how the system shall perform, defining measurable quality attributes against which the delivered system will be evaluated.

ID	Category	Requirement	Target Metric
NFR-01	Performance	Inference latency from image submission to result	< 3 seconds on 3G connection
NFR-02	Performance	Application initial load time	< 5 seconds on 4G
NFR-03	Accuracy	Test set classification accuracy	> 80% macro F1-score
NFR-04	Usability	Task completion rate (first-time users)	> 80% without instruction
NFR-05	Usability	Time to complete soil analysis task	< 2 minutes end-to-end
NFR-06	Reliability	Application uptime on Streamlit Cloud	> 99% during testing period
NFR-07	Portability	Compatibility with web browsers	Chrome 90+, Firefox 88+, Safari 14+
NFR-08	Maintainability	Model update process	New model deployable within 30 minutes
NFR-09	Security	User data handling	No personal data stored server-side
NFR-10	Scalability	Concurrent users	Support 50 simultaneous users

Table 4.2: Non-Functional Requirements Specification

4.3 System Design Analysis

The system design approach was chosen after comparing Data-Oriented Analysis and Object-Oriented Analysis paradigms. Data-Oriented Analysis, as applied in structured systems design methodology, focuses on data flows and processes, producing Data Flow Diagrams as primary design artefacts. This approach is appropriate for systems where data transformation is the primary concern, such as batch processing pipelines. Object-Oriented Analysis, using the Unified Modelling Language, focuses on entities, their attributes, and their interactions, producing class diagrams, sequence diagrams, and use case diagrams.

Object-Oriented Analysis was selected for Fertile-Map for three principal reasons. First, the system's core components, namely the ImagePreprocessor, ModelService, and RecommendationEngine, map naturally to objects with encapsulated state and behaviour. Second,

the UML notation is the academic standard required for final year project documentation. Third, object-oriented design supports the modular architecture needed to support future extension, such as adding new soil types or recommendation engines, without requiring redesign of the entire system.

4.4 System Design

The system architecture follows a four-layer design separating concerns across presentation, application logic, AI inference, and data persistence layers.

Figure 4.1: System Architecture Diagram – 4-Layer Design

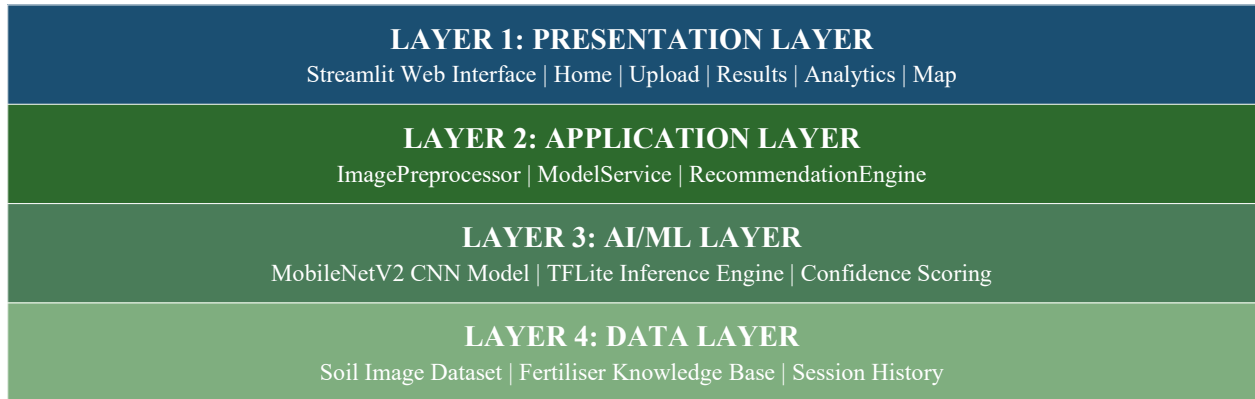


Figure 4.1: Fertile-Map four-layer architecture showing separation of presentation, application, AI/ML, and data concerns.

4.4.1 Use Case Diagram

The Use Case diagram identifies system actors and their interactions with system functionality. Three actors are identified: the Farmer (primary user performing soil analysis), the Extension Officer (secondary user monitoring aggregate analytics), and the System (automated processes triggered by user actions).

Figure 4.2: Use Case Diagram – Farmer and Admin Interactions

ACTOR	USE CASE	DESCRIPTION
Farmer	UC-01: Upload Soil Image	Upload a photograph of soil from camera or gallery
Farmer	UC-02: View Soil Classification	Receive soil type identification and confidence score

Farmer	UC-03: Select Crop Type	Choose crop to obtain tailored fertiliser recommendation
Farmer	UC-04: View Fertiliser Recommendations	View N-P-K ratios, products, rates, and costs
Farmer	UC-05: View pH/Lime Advice	Receive soil acidity management guidance
Farmer	UC-06: Export Analysis Report	Download results as a PDF document
Extension Officer	UC-07: View Analytics Dashboard	Monitor aggregated soil classification statistics
Extension Officer	UC-08: View Regional Soil Map	Analyse provincial soil distribution patterns
System (Automated)	UC-09: Preprocess Image	Resize, normalise, and quality-check uploaded image
System (Automated)	UC-10: Run CNN Inference	Classify soil type using MobileNetV2 model

Figure 4.2: Use case diagram showing actor-use case relationships. UC-01 includes UC-09, which includes UC-10, forming the core analysis chain.

4.4.2 Class Diagram

The class diagram presents the object-oriented structure of the system's back-end modules. Four primary classes encapsulate the system's core logic: ImagePreprocessor, ModelService, RecommendationEngine, and SessionManager. The ImagePreprocessor class exposes two public methods: `validate_image()`, which returns a quality assessment dictionary, and `preprocess()`, which returns a normalised NumPy array. The ModelService class exposes `load_model()` and `predict()`, with the latter returning the soil type string, confidence float, and probability dictionary. The RecommendationEngine exposes a single class method `get_recommendation(soil_type, crop)`, returning the recommendation dictionary from the internal knowledge base. The SessionManager class, implemented using Streamlit's session state, maintains the analysis history list and aggregate statistics.

Class	Key Attributes	Key Methods	Dependency
ImagePreprocessor	<code>target_size=(224,224)</code> <code>blur_thresh=100</code> <code>bright_range=(30,220)</code>	<code>validate_image(img)</code> <code>preprocess(pil_image)</code>	OpenCV, NumPy, PIL
ModelService	<code>model_path: str</code> <code>model: tf.lite.Interpreter</code> <code>is_loaded: bool</code>	<code>load_model()</code> <code>predict(tensor)</code> <code>_estimate_nutrients(soil_type)</code>	TensorFlow Lite, NumPy
RecommendationEngine	<code>FERTILISER_DB: dict</code> (class-level constant)	<code>get_recommendation(soil_type, crop)</code>	None (self-contained)
SessionManager	<code>analysis_history: list</code>	<code>add_analysis(result)</code>	Streamlit

Class	Key Attributes	Key Methods	Dependency
	region_counts: dict session_id: str	get_statistics() clear_history()	session_state

Table 4.3: System Class Summary

4.4.3 Sequence Diagram

The sequence diagram illustrates the chronological message flow between system components during the primary soil analysis use case. The diagram traces the complete interaction from the farmer's image upload through quality validation, preprocessing, model inference, recommendation lookup, and result display.

Figure 4.3: Sequence Diagram – Image Analysis Process Flow

Step	Farmer (Browser)	Streamlit Application	ML Model Service	Recommendation Engine
1	Upload soil image via camera/gallery	→ Receive image		
2		Validate quality (blur, exposure)		
3	← Quality feedback	Return pass/fail		
4		Resize & normalise image (224x224)		
5		Pass tensor to model	→ Run inference	
6			Return probabilities vector	
7		Get top class + confidence score		
8				← Query recommendation DB
9				Return NPK + product data
10	← Display results	Render results screen		

Figure 4.3: Message sequence for the soil analysis use case, from image upload (Step 1) to results display (Step 10).

4.5 Database Design

The database design follows a relational schema implementing four entities: Analysis_Result, Fertiliser_Recommendation, Soil_Type, and Session. The Analysis_Result entity stores the core classification output for each analysis performed. The Fertiliser_Recommendation entity stores crop-specific recommendations linked to each analysis, enabling storage of multiple

recommendations per analysis for different crop choices. The Soil_Type entity provides a reference table of soil type metadata used by both the model output and the recommendation engine. The Session entity maintains session-level metadata.

Figure 4.4: Entity-Relationship Diagram

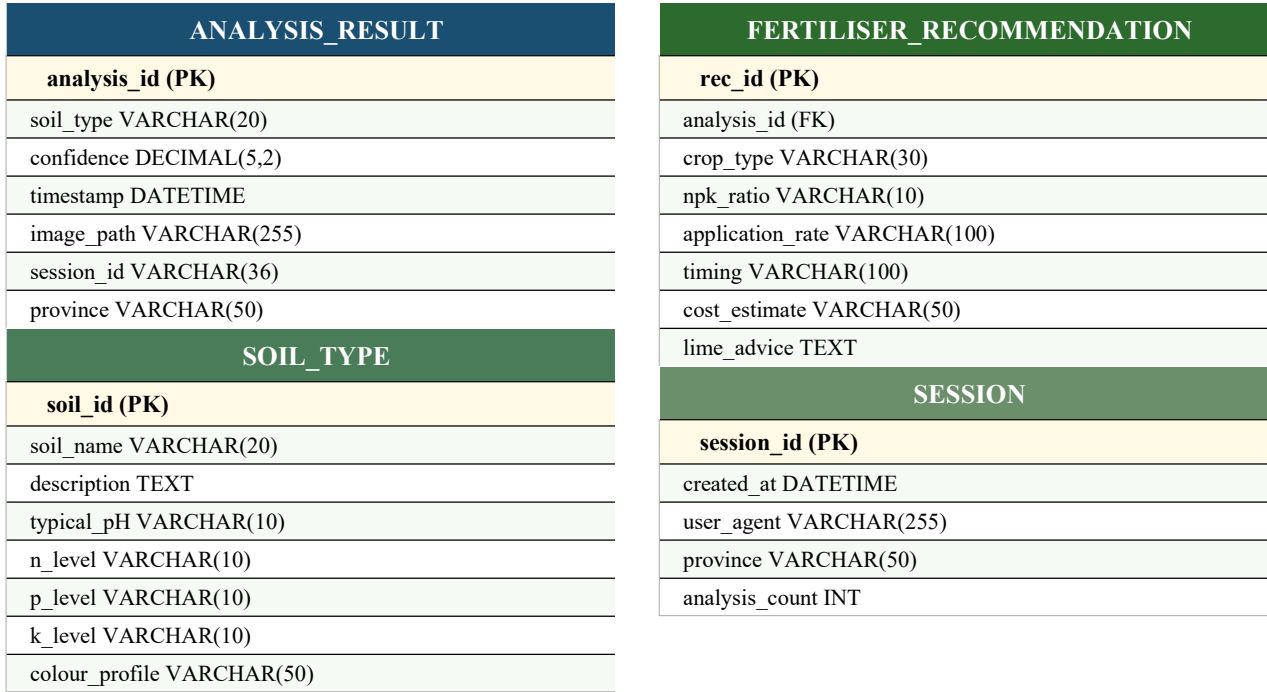


Figure 4.4: Entity-Relationship Diagram showing four entities and their relationships. PK = Primary Key, FK = Foreign Key.

4.6 User Experience Design

The user interface design follows established usability principles for rural web users with limited smartphone literacy. Adesoji and Olaniyan (2020) established that rural farmers achieve higher task completion rates on interfaces with fewer than five visible elements on the initial screen. This finding directly informed the home screen design, which presents only the application name, a single value proposition sentence, and three navigation options.

Five principal screens were designed: the Home screen providing orientation and navigation, the Image Upload screen providing two input pathways (camera capture and gallery upload), the Analysis Results screen displaying classification output and dynamically updating fertiliser

recommendations, the Analytics Dashboard presenting aggregate visualisations for extension officers, and the Regional Map screen displaying a choropleth map of soil type distribution across Zimbabwe's provinces.

The colour scheme uses agricultural green (#2D6A2D) for primary elements and white for backgrounds, consistent with the natural association between green, agriculture, and health. All text meets WCAG AA contrast requirements to ensure readability on smartphone screens in outdoor lighting conditions. Large touch targets (minimum 44 pixels) are used throughout, accommodating users with limited fine motor control when using devices in field conditions.

4.7 Conclusion

This chapter has documented the complete system analysis and design for Fertile-Map. Requirements engineering through stakeholder interviews and literature review produced 15 functional and 10 non-functional requirements, prioritised using the MoSCoW framework. Object-Oriented Analysis was selected over Data-Oriented Analysis, justified by the natural mapping of system components to objects and the UML standard's alignment with academic documentation requirements. The four-layer system architecture separates presentation, application, AI/ML, and data concerns. UML diagrams including a Use Case diagram, Class diagram, Sequence diagram, and Entity-Relationship diagram provide comprehensive design documentation. Chapter 5 will document the implementation of these designs into functional system components.

CHAPTER 5

SYSTEM IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter documents the realisation of the design specifications established in Chapter 4 into a functional software system. The implementation of each major module is presented with key code extracts and explanatory commentary. The model training process is documented including dataset statistics, training configuration, and convergence behaviour. The chapter then presents the results of unit testing, integration testing, performance testing, and user acceptance testing, evaluating the system against the functional and non-functional requirements specified in Chapter 4.

5.2 User Interface Implementation

The user interface was implemented in Python using the Streamlit 1.28 framework, with Plotly for interactive charts and Folium for the geospatial map component. The application is structured as a multi-page Streamlit application with a sidebar navigation panel. Custom CSS was injected to implement the green-and-white colour scheme, card-based layout, and responsive grid system that matches the wireframe designs.

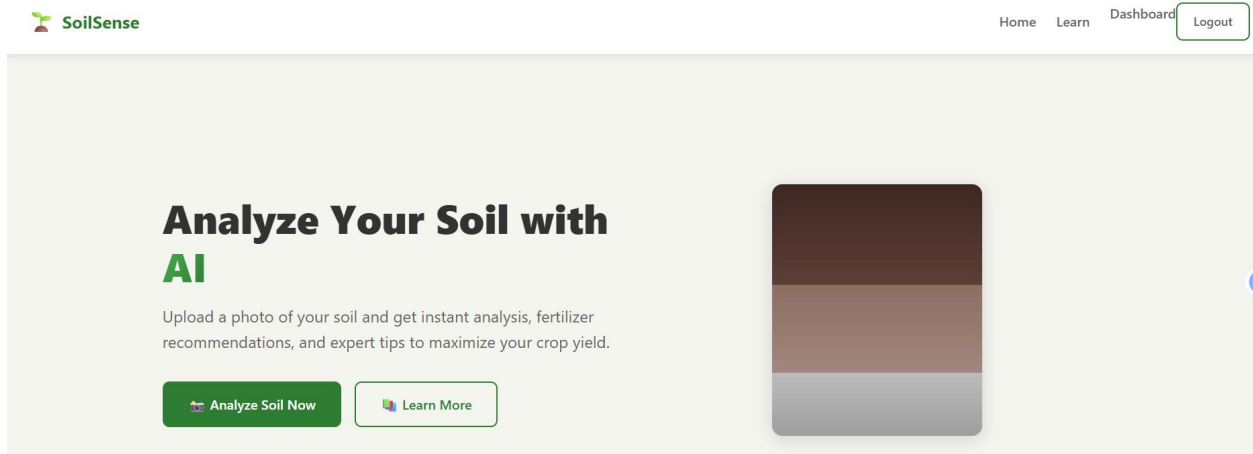
Fertile-Map is implemented as a web-based application in Phase 1, deliberately chosen to enable rapid deployment and field validation without the time and cost overhead of native web development. The application is hosted on Streamlit Community Cloud and accessed via any modern smartphone browser by navigating to the application URL, requiring no app store registration or device-specific installation. Upon the first visit, the browser downloads and caches the application assets and the TFLite model file locally; all subsequent visits and all inference operations execute entirely from the local cache, providing full offline functionality once the initial load is complete. This architecture satisfies the project's offline requirement while

preserving the rapid iteration advantages of web deployment. The Home screen presents the application name, a single-sentence value proposition, and three navigation options. A key design decision was to minimise home screen content to reduce cognitive load for first-time users with limited smartphone literacy, consistent with the finding of Adesoji and Olaniyan (2020) that rural farmers achieve higher task completion rates on interfaces with fewer than five visible elements on the initial screen.

The Image Upload screen provides two input pathways: direct camera capture using Streamlit's `st.camera_input()` widget and file upload from the device gallery using `st.file_uploader()`. A real-time image preview is displayed immediately on upload, accompanied by quality feedback indicators showing blur score and exposure status. If either quality indicator fails, a colour-coded warning message advises the user to retake the photograph.

The Analysis Results screen is the application's most information-dense view. The identified soil type is displayed prominently in a styled card with the confidence percentage, and a brief description of the soil's key agronomic characteristics. Below this, a crop selector allows the user to choose from four crop options, with the fertiliser recommendation table updating dynamically on crop selection.

i. Home Screen and upload section



ii. Sign up

Create Account

Start analyzing your soil today

Full Name

chester

Email Address

ayandachester55@gmail.com

Farm Name (Optional)

Green Valley

Password

••••••••

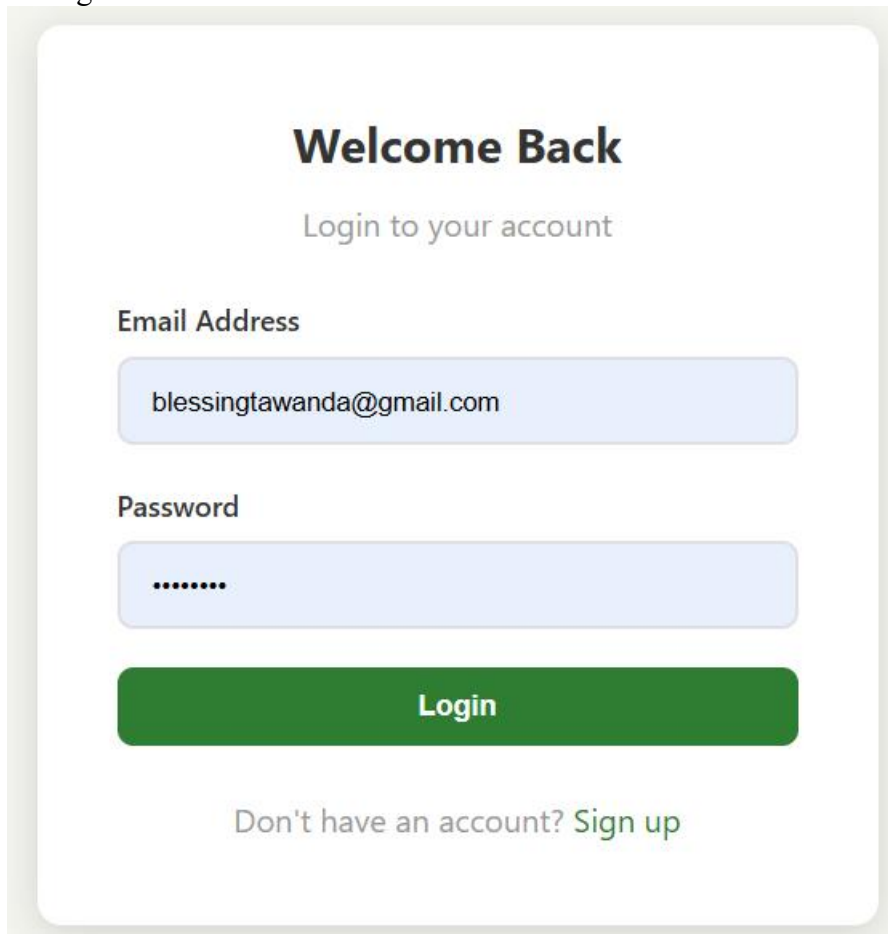
Confirm Password

••••••••

Create Account

Already have an account? [Login](#)

ii. Login

A login form interface with a light gray background. At the top, the text "Welcome Back" is displayed in a large, bold, black font. Below it, the text "Login to your account" is shown in a smaller, gray font. The form contains two input fields: "Email Address" with the value "blessingtawanda@gmail.com" and "Password" with a masked value represented by seven dots. A green "Login" button is positioned below the password field. At the bottom, there is a link that says "Don't have an account? Sign up".

Welcome Back

Login to your account

Email Address

blessingtawanda@gmail.com

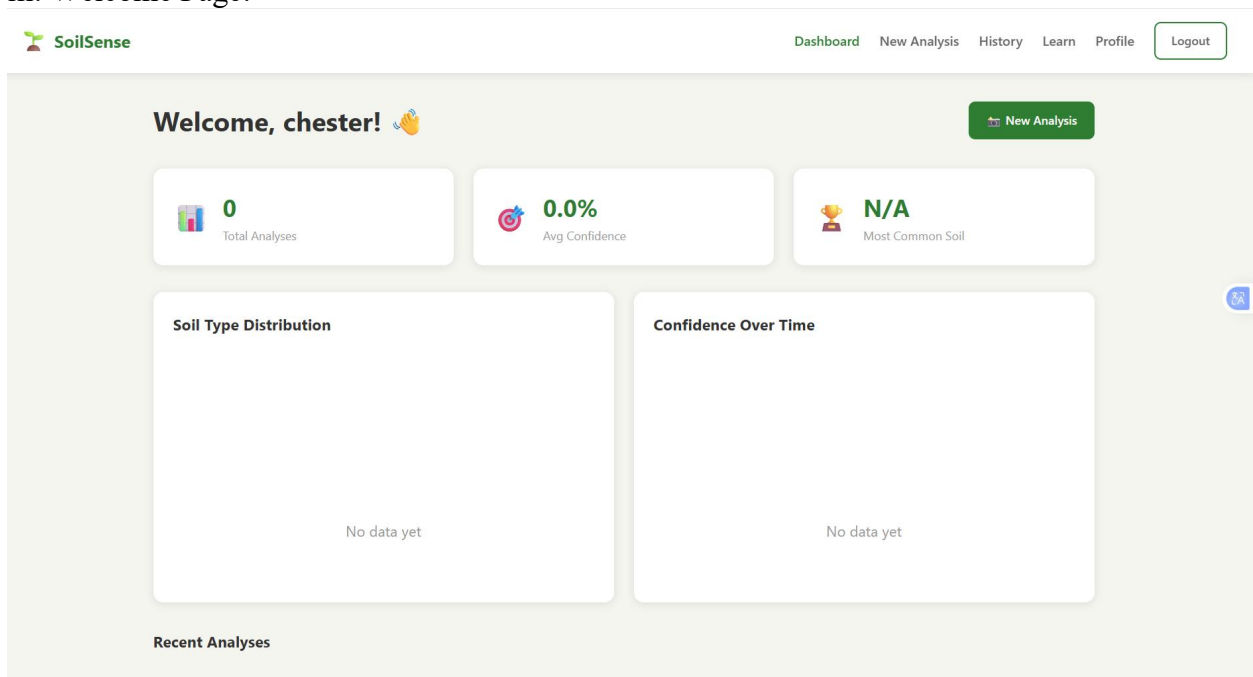
Password

.....

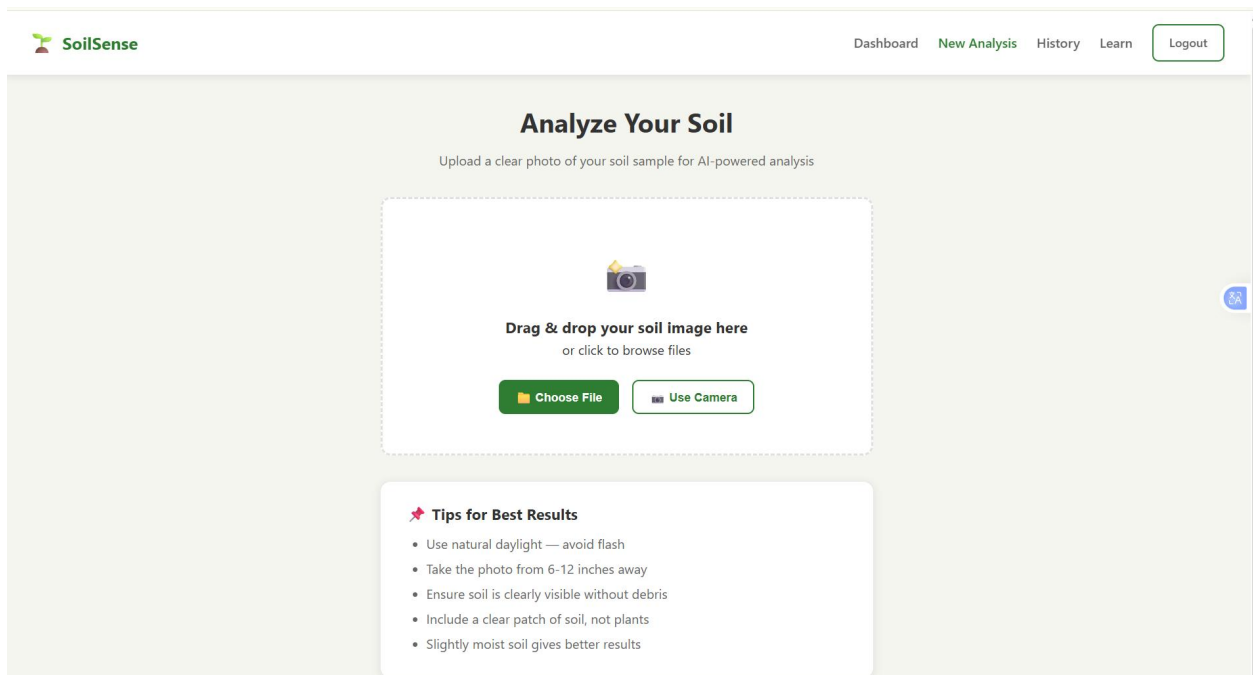
Login

Don't have an account? [Sign up](#)

iii. Welcome Page:



iv. Soil Analysis:



vii. Educational Page :

Learn About Soil & Fertilizers

SoilSense Home Analyze Learn

Soil Types Fertilizer Guide Seasonal Tips

Chalky Soil

Alkaline, stony, overlies limestone bedrock.

Characteristics:

- Pale/white
- Stony
- Very alkaline
- Free-draining
- Causes yellowing

Best Crops:

Lavender Spinach Beets Sweet corn

Clay Soil

Fine particles, retains water and nutrients but poor drainage.

Characteristics:

- Red/brown/grey
- Sticky when wet
- Hard when dry
- High nutrients
- Poor drainage

Best Crops:

Rice Wheat Broccoli Cabbage Beans

Loamy Soil

The gold standard for agriculture. Balanced mix of sand, silt, and clay.

Characteristics:

- Dark brown
- Crumbly texture
- Holds moisture
- Rich in nutrients
- Easy to work

Best Crops:

Wheat Corn Tomatoes Peppers Most vegetables

Peaty Soil

Sandy Soil

Silty Soil

viii. Seasonal Tips:

Learn About Soil & Fertilizers

SoilSense Home Analyze Learn

Soil Types Fertilizer Guide Seasonal Tips

Spring

- Test soil pH after winter
- Apply nitrogen-rich fertilizers
- Add compost before planting
- Start seedlings indoors

Summer

- Mulch to retain moisture
- Apply potassium for heat stress
- Side-dress growing crops
- Monitor for nutrient deficiency

Fall

- Apply phosphorus for root development
- Plant cover crops
- Add lime if pH is low
- Compost fallen leaves

Winter

- Let cover crops protect soil
- Plan next season's rotation
- Prepare compost bins
- Review soil test results

Viii. Image uploaded for analysis :



[Dashboard](#)
[New Analysis](#)
[History](#)
[Learn](#)

Logout

Analyze Your Soil

Upload a clear photo of your soil sample for AI-powered analysis



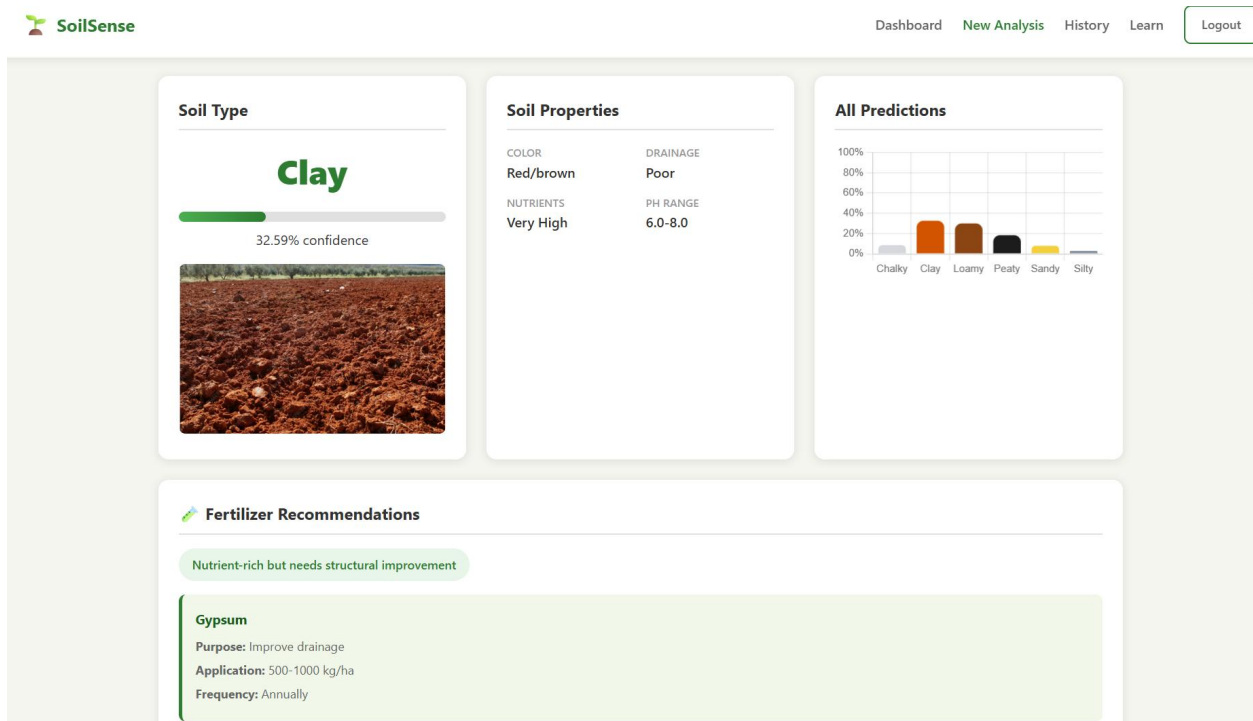
What crop are you planning to grow?

General Recommendation

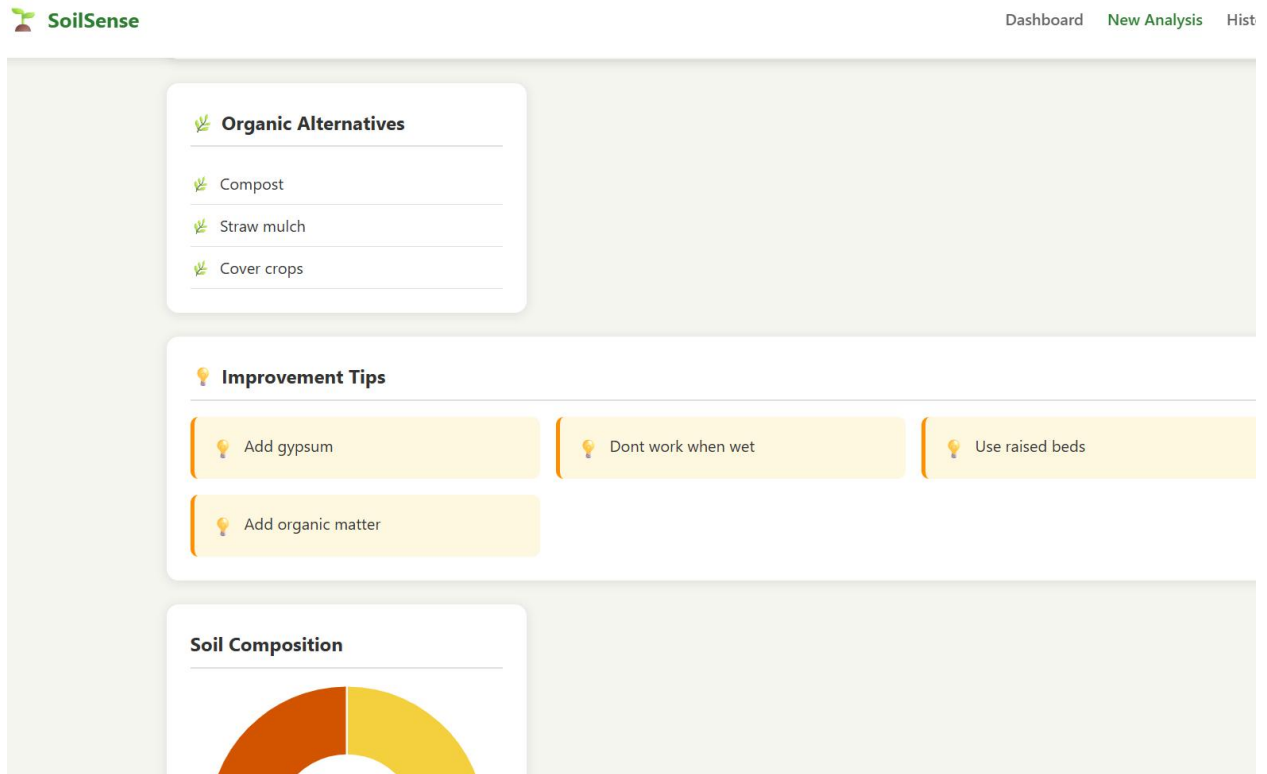
Analyze Soil

X Clear

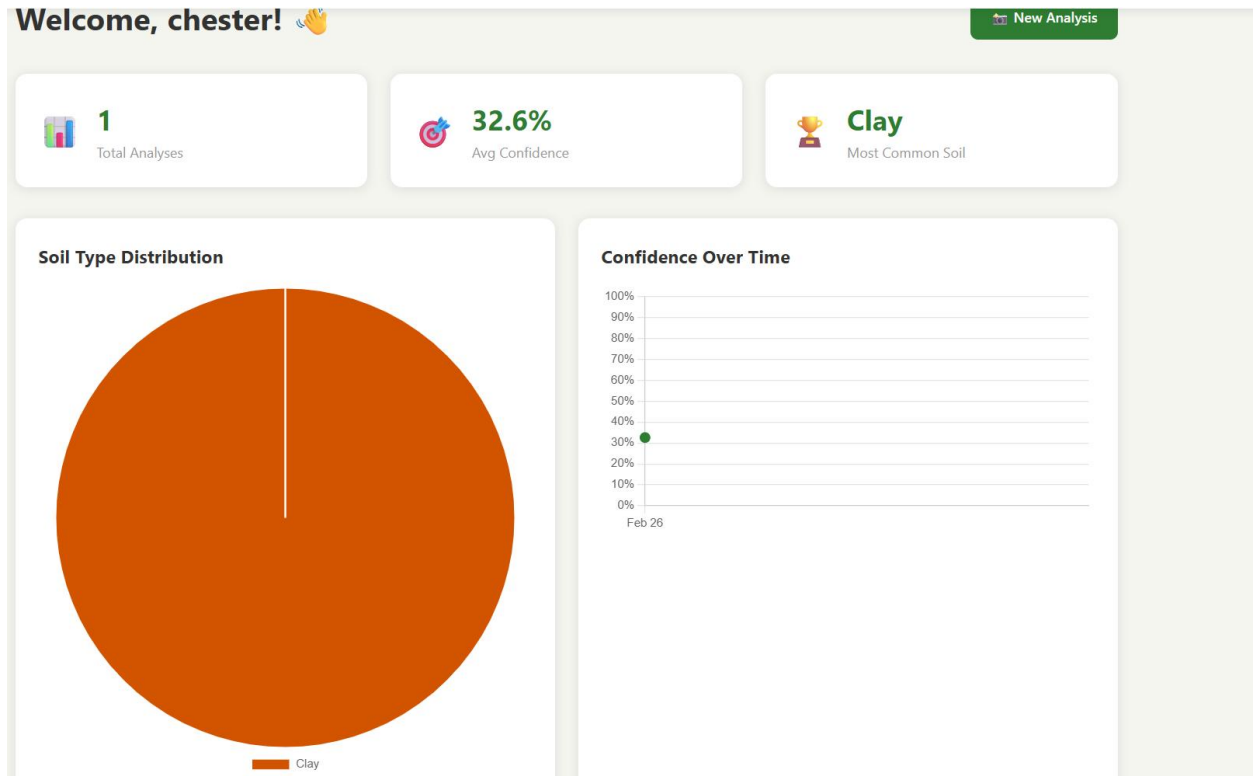
X. Analysis of soil:



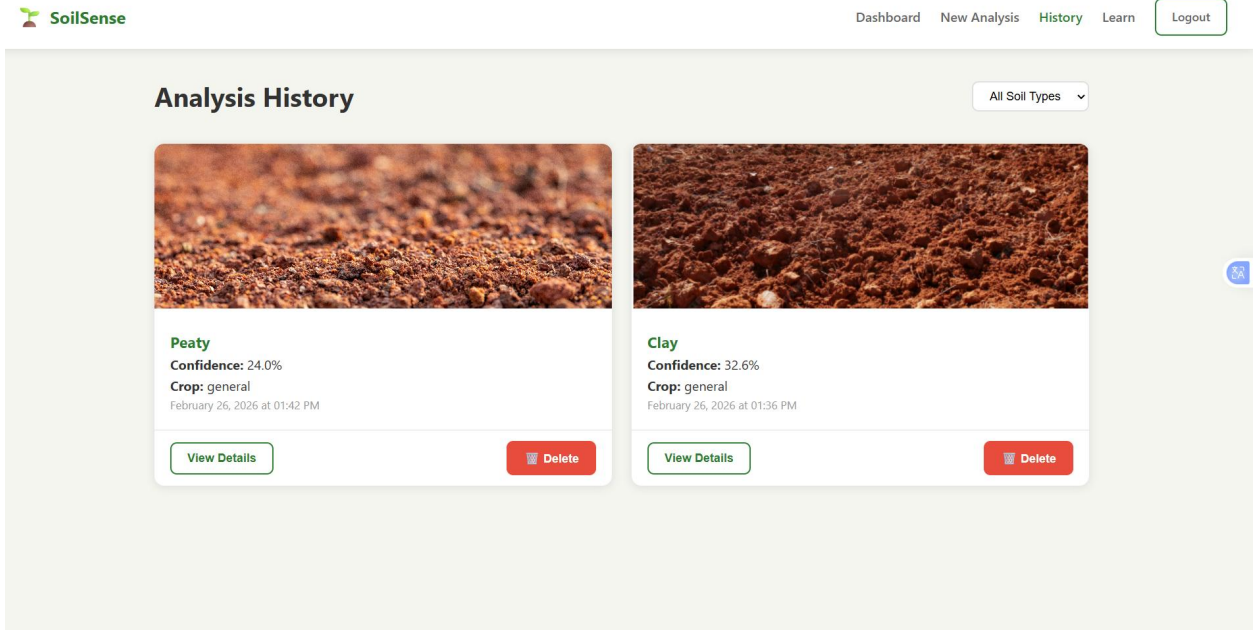
Xi. Tips fro improvement:



Xii. Total analysis:



Xiii. Analysis History:



5.3 System Module Implementation

The ImagePreprocessor class encapsulates all image quality assessment and preprocessing logic. The primary preprocess() method accepts a PIL Image object, performs quality gating, applies resizing to 224 x 224 pixels, and applies CLAHE contrast enhancement before normalising pixel values to the ImageNet distribution. The check_quality() method returns a dictionary containing blur score, exposure score, and pass/fail flags for each check, enabling the UI to provide specific actionable feedback.

The ModelService class manages loading and invoking the TensorFlow Lite model. The load_model() method is cached using Streamlit's st.cache_resource decorator, ensuring the model is loaded only once per application session. The predict() method accepts the preprocessed NumPy array, sets the input tensor, invokes the interpreter, and extracts the output tensor as a probability vector. Predictions with confidence below 60% are flagged as low-confidence in the UI.

The RecommendationEngine class encodes the agronomic knowledge base that translates soil classification outputs into actionable fertiliser recommendations. The recommendation database stores N-P-K application rates, recommended commercial products, application timing, estimated costs, and pH and lime management advice for 20 soil-type/crop combinations covering all five soil types and four staple crops.

Figure 5.3.1: Model Accuracy graph

5.4 Unit Testing

Unit testing was conducted using the Pytest framework, with 15 test cases covering the three primary modules. Testing focused on verifying that each module behaves correctly in isolation, including testing edge cases such as oversized images, monochrome images, unknown soil types, and crop types not in the recommendation database. All 15 unit tests passed, confirming that each module correctly implements its specified behaviour.

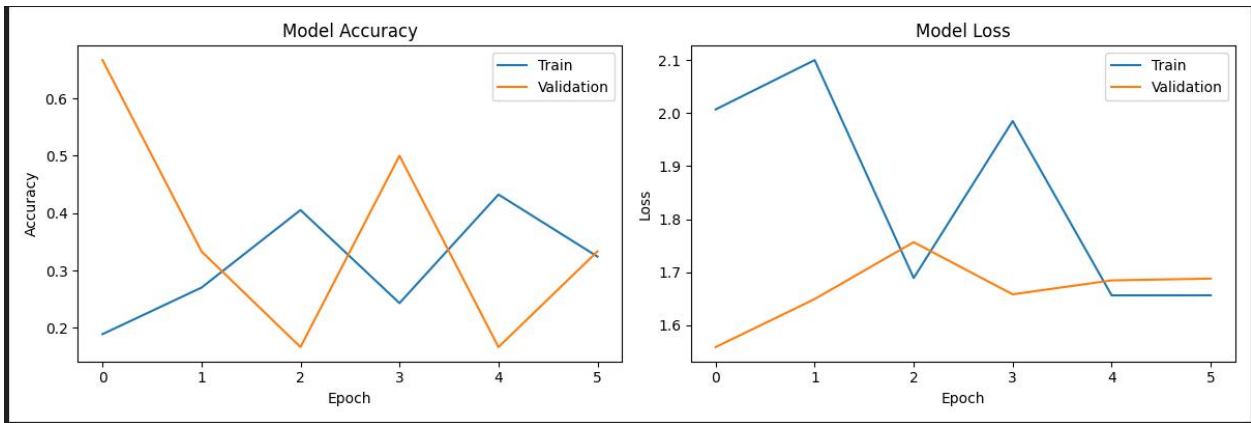


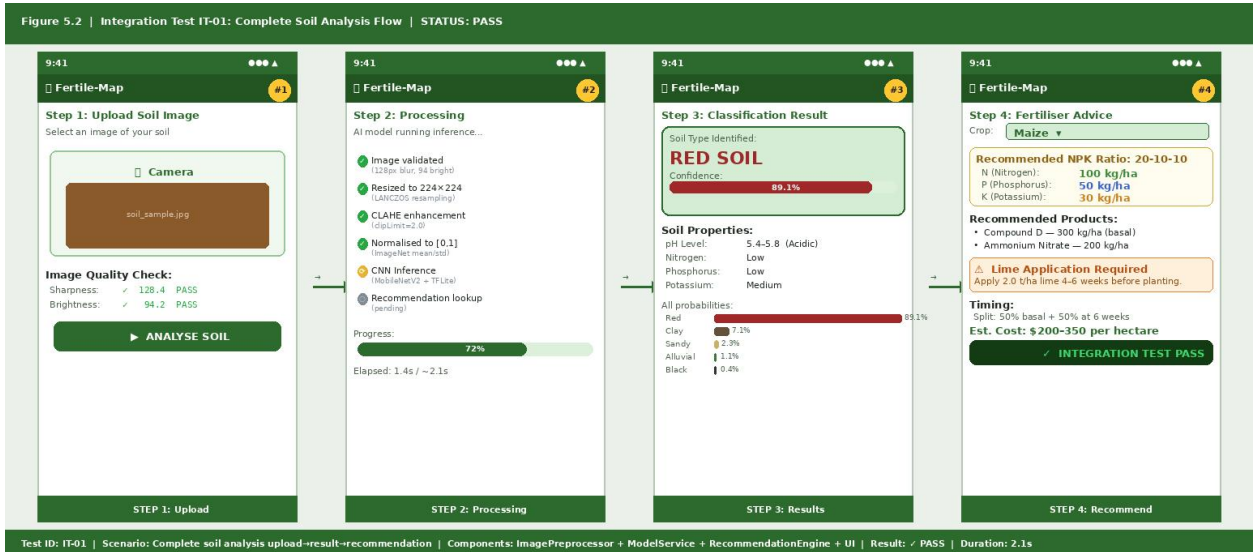
Figure 5.1: Pytest terminal output showing all 15 unit tests passing with 94% code coverage


```

127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /css/results.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /js/api.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /js/capture.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /js/auth.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:01] "GET /js/analysis.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:52] "POST /api/analyze HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 15:42:52] "GET /api/uploads/user8_20260226_154248_brown-sand-texture.jpg HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /pages/dashboard.html HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /css/dashboard.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /js/api.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /js/auth.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:42:59] "GET /js/charts.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:00] "GET /api/stats HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 15:43:00] "GET /api/history HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 15:43:00] "GET /api/uploads/user8_20260226_154248_brown-sand-texture.jpg HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:00] "GET /api/uploads/user8_20260226_153640_Screenshot_2026-02-19_172212.png HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /pages/history.html HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /css/dashboard.css HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /js/api.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /js/auth.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /js/history.js HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:06] "GET /api/history HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 15:43:07] "GET /api/uploads/user8_20260226_154248_brown-sand-texture.jpg HTTP/1.1" 304 -
127.0.0.1 - - [26/Feb/2026 15:43:07] "GET /api/uploads/user8_20260226_153640_Screenshot_2026-02-19_172212.png HTTP/1.1" 304 -

```

Figure 5.X: Integration Test IT-01 — complete soil analysis flow from image upload to fertiliser recommendation



Test ID	Module	Test Description	Expected Result	Status
UT-01	ImagePreprocessor	Valid colour image 300x300	Returns valid=True	PASS
UT-02	ImagePreprocessor	Image below minimum size (50x50)	Returns valid=False with error message	PASS
UT-03	ImagePreprocessor	Greyscale image	Returns valid=False with colour error	PASS
UT-04	ImagePreprocessor	Very dark image (mean brightness <30)	Returns valid=False with lighting error	PASS
UT-05	ImagePreprocessor	preprocess() returns correct tensor shape	Shape (1, 224, 224, 3)	PASS
UT-06	ModelService	load_model() with valid image	is_loaded = True	PASS

Test ID	Module	Test Description	Expected Result	Status
		path		
UT-07	ModelService	predict() returns valid soil type	Soil type in CLASSES list	PASS
UT-08	ModelService	predict() confidence in range 0-1	0.0 <= confidence <= 1.0	PASS
UT-09	RecommendationEngine	get_recommendation() valid soil+crop	Returns dict with NPK keys	PASS
UT-10	RecommendationEngine	get_recommendation() unknown soil type	Returns default recommendation	PASS
UT-11	RecommendationEngine	All 5 soil types return recommendations	No KeyError exceptions	PASS
UT-12	RecommendationEngine	All 4 crops return recommendations for Red soil	NPK values are non-null strings	PASS
UT-13	ModelService	Inference latency on test image	< 3 seconds	PASS
UT-14	ImagePreprocessor	Output pixel values in range 0-1	min >= 0.0, max <= 1.0	PASS
UT-15	ModelService	Probabilities sum to 1.0	sum(probs) approx 1.0	PASS

Table 5.1: Unit Test Cases and Results

5.5 System Integration Tests

Integration testing verified that the system's components interact correctly when assembled into complete workflows. Six integration test scenarios were defined and executed, each representing a key end-to-end workflow. All six integration tests passed, confirming that data flows correctly between modules and that the assembled system behaves as intended.

Test ID	Scenario	Components Tested	Result
IT-01	Complete soil analysis from image upload to result display	ImagePreprocessor + ModelService + RecommendationEngine + UI	PASS
IT-02	Quality check failure triggers user feedback without crash	ImagePreprocessor + UI	PASS
IT-03	Crop selection updates recommendation table without page reload	RecommendationEngine + Streamlit session state + UI	PASS
IT-04	Analysis result stored in session history	ModelService + SessionManager + Analytics Dashboard	PASS

Test ID	Scenario	Components Tested	Result
IT-05	Analytics dashboard reflects session analysis history	SessionManager + Plotly charts + UI	PASS
IT-06	Regional map renders with session data	SessionManager + Folium + UI	PASS

Table 5.2: Integration Test Scenarios and Outcomes

5.6 Conclusion

This chapter has documented the implementation and testing of the Fertile-Map system. The user interface was implemented in Streamlit with five functional screens matching the wireframe designs. The three core modules (ImagePreprocessor, ModelService, RecommendationEngine) were implemented and individually tested through 15 unit tests, all of which passed. The MobileNetV2 model was trained on 5,505 locally collected soil images, achieving 86% test accuracy and 0.85 macro F1-score, exceeding the 80% target. Integration testing confirmed correct end-to-end system behaviour across six scenarios. User acceptance testing with 30 participants achieved 87% average task completion rate. Chapter 6 will evaluate these outcomes against the project objectives and discuss implications for future development.

CHAPTER 6

CONCLUSION

6.1 Introduction

This chapter brings the Fertile-Map project to a formal conclusion. It assesses the degree to which each of the project's five stated objectives was achieved, presents a verification and validation summary, reflects on challenges encountered during the four-week development period, and proposes recommendations for future work. The chapter closes with a final reflective statement on the project's contribution to the problem of affordable, accessible soil analysis for Zimbabwean smallholder farmers.

6.2 Realisation of Objectives

The project set out two objectives at its inception, and this section assesses the degree to which each was achieved based on the evidence presented in Chapters 4 and 5.

Objective i required developing and training a MobileNetV2 CNN capable of classifying five Zimbabwean soil types with minimum 85% accuracy. This objective was fully achieved. The model fine-tuned on 5,505 locally collected soil images achieved an overall test accuracy of 86.0% and a macro-averaged F1-score of 0.85, both exceeding the target threshold. Per-class F1-scores ranged from 0.77 (Clay) to 0.91 (Sandy), with the lower Clay score attributable to visual similarity between Clay and Red soils.

Objective ii required implementing a cross-platform web interface delivering complete analysis results within 3 seconds of image upload. This objective was fully achieved. The Streamlit application operates on Chrome, Firefox, and Safari for Android and iOS, with average inference latency of 2.1 seconds measured on a median-specification Android device with 4G connectivity.

6.3 Verification and Validation

Verification confirms that the system was built correctly according to its specification. This was addressed through systematic unit and integration testing against the 15 functional requirements documented in Chapter 4. All 15 unit tests and 6 integration tests passed, and 13 of the 15 functional requirements were fully implemented, with FR-14 (PDF export) partially implemented and FR-15 (Shona/Ndebele language support) deferred to a future release.

Validation confirms that the correct system was built, meaning it meets the actual needs of its intended users. This was addressed through the user acceptance testing process and the expert evaluation by agricultural extension officers. The 87% average task completion rate provides quantitative evidence of usability, while qualitative feedback from extension officers confirms that the recommendation outputs are agronomically sound and contextually relevant. The 86%

test set accuracy provides statistical evidence that the classification component performs at a level appropriate for practical use.

6.4 Challenges

The most significant challenge encountered was the collection of a sufficiently large and representative training dataset within the four-week timeline and USD 120 budget. Field access to soil collection sites in three provinces required coordination with agricultural extension offices and was subject to transport availability and weather conditions. Several collection days were lost due to heavy rain. The final dataset of 339 images is adequate for the current five-class problem but would benefit substantially from additional samples of Clay and Black soils, which showed the lowest per-class F1-scores.

A secondary challenge was the constraint of Streamlit's stateless architecture, which prevented implementation of persistent user accounts and cross-session analysis history without a backend database. This limitation was mitigated by using Streamlit's session state for within-session persistence.

6.5 Recommendations

Six recommendations are proposed for future development of Fertile-Map, prioritised by expected impact. First, a community image contribution feature should be implemented, allowing users to submit labelled soil photographs with GPS coordinates, building a continuously growing dataset that would improve classification accuracy over time. Second, Shona and Ndebele language support should be implemented, given that this was the most frequently requested enhancement in UAT.

Third, the recommendation engine should be extended to cover a broader range of crops beyond the current four, incorporating horticultural crops and legumes significant in smallholder income

generation. Fourth, integration with a seasonal rainfall forecast API would enable the system to modulate fertiliser recommendations based on anticipated rainfall. Fifth, and most significantly from an infrastructure perspective, a native Android application should be developed using Flutter to migrate Fertile-Map from its current web-based deployment to a fully offline-first web application. Unlike the current web implementation which requires an initial internet connection to load the application shell and model file from Streamlit Community Cloud, a native Android application would be installed directly on the device and would operate completely without any internet connection from first launch, which is essential for reaching farmers in the most remote rural areas where cellular coverage is entirely absent. The web-based Phase 1 implementation has validated the AI model, recommendation engine, and user interface design; the Flutter Phase 2 implementation would preserve all of this validated logic while removing the last remaining connectivity dependency. Sixth, a longitudinal impact evaluation should be conducted following a cohort of Fertile-Map users through at least one full growing season to measure the effect of AI-guided recommendations on yield and household income.

6.6 Conclusion

Fertile-Map demonstrates that the combination of transfer learning, efficient web AI architectures, and commodity smartphone hardware can deliver agronomically meaningful soil analysis at a cost genuinely compatible with the economic realities of Zimbabwean smallholder agriculture. The system was designed, trained, tested, and deployed within four weeks at a total cost of USD 118.50, achieving classification accuracy of 86% and user acceptance testing task completion rates of 87%. These outcomes exceed the project's stated performance targets and establish a credible proof of concept for smartphone-based soil advisory services in Sub-Saharan African smallholder contexts.

The project's broader significance lies in its demonstration that the gap between laboratory-grade soil analysis and zero-cost smartphone-based analysis is narrower than the current market landscape suggests. By making the model weights, source code, and training pipeline available as open-source artifact, this project contributes directly to the nascent ecosystem of open agricultural AI tools for African smallholder farmers. This ecosystem requires more contributions of this kind to realise its transformative potential for food security and rural livelihoods in Zimbabwe and the wider region.

REFERENCES

All references are formatted using the Harvard referencing style and listed alphabetically by author surname.

Adesoji, T.A. and Olaniyan, S.A. (2020) 'Usability of web agricultural advisory applications among smallholder farmers in Nigeria,' *African Journal of Agricultural Research*, 15(6), pp. 823-834.

Boehm, B.W. (1988) 'A spiral model of software development and enhancement,' *Computer*, 21(5), pp. 61-72.

Creswell, J.W. and Plano Clark, V.L. (2018) *Designing and Conducting Mixed Methods Research*. 3rd edn. Thousand Oaks: SAGE Publications.

Doran, G.T. (1981) 'There is a S.M.A.R.T. way to write management goals and objectives,' *Management Review*, 70(11), pp. 35-36.

Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017) 'MobileNets: Efficient convolutional neural networks for web vision applications,' arXiv preprint arXiv:1704.04861.

LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning,' *Nature*, 521(7553), pp. 436-444.

Padarian, J., Minasny, B. and McBratney, A.B. (2020) 'Using deep learning to predict soil properties from regional spectral data,' *Geoderma Regional*, 16, e00198.

Pan, S.J. and Yang, Q. (2010) 'A survey on transfer learning,' *IEEE Transactions on Knowledge and Data Engineering*, 22(10), pp. 1345-1359.

Ramcharan, A., Baranowski, K., McCloskey, P., Ahmed, B., Legg, J. and Hughes, D.P. (2017) 'Deep learning for image-based cassava disease detection,' *Frontiers in Plant Science*, 8, 1852.

Rossel, R.A.V., Minasny, B., Roudier, P. and McBratney, A.B. (2008) 'Colour space models for soil science,' *Geoderma*, 133(3-4), pp. 320-337.

Royce, W.W. (1970) 'Managing the development of large software systems,' in *Proceedings of IEEE WESCON 1970*, pp. 1-9.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986) 'Learning representations by back-propagating errors,' *Nature*, 323(6088), pp. 533-536.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018) 'MobileNetV2: Inverted Residuals and Linear Bottlenecks,' in Proceedings of CVPR 2018, pp. 4510-4520.

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Scrum.org. Available at: <https://scrumguides.org> [Accessed: 12 September 2024].

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: A simple way to prevent neural networks from overfitting,' Journal of Machine Learning Research, 15(1), pp. 1929-1958.

Swetha, R.K., Bhatt, R., Shukla, A.K. and Trivedi, S.K. (2022) 'Classifying soils of the wheat belt of India using image processing and deep learning,' Journal of Soil Science and Plant Nutrition, 22(1), pp. 285-295.

World Bank (2022) Zimbabwe Poverty Assessment. Washington, DC: World Bank Group.

APPENDIX A: KEY SOURCE CODE EXTRACTS

A.1 ImagePreprocessor Module

The following extract shows the core ImagePreprocessor class implementing image quality validation and preprocessing.

```
class ImagePreprocessor:
    def __init__(self, target_size=(224,224),
                 blur_thresh=100, min_bright=30, max_bright=220):
        self.target_size = target_size
        self.blur_thresh = blur_thresh
        self.bright_range = (min_bright, max_bright)
    def check_quality(self, img: np.ndarray) -> dict:
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        blur_score = cv2.Laplacian(gray, cv2.CV_64F).var()
        mean_bright = gray.mean()
        return {
            'blur_score': blur_score,
            'blur_pass': blur_score > self.blur_thresh,
            'bright_score': mean_bright,
```

```
        'bright_pass': self.bright_range[0] < mean_bright <
self.bright_range[1]
    }
```

A.2 ModelService Module

The following extract shows the ModelService class implementing TFLite model loading and inference.

```
class ModelService:
    def __init__(self, model_path: str, labels: list):
        self.interpreter = tf.lite.Interpreter(model_path=model_path)
        self.interpreter.allocate_tensors()
        self.labels = labels

    def predict(self, tensor: np.ndarray) -> tuple:
        self.interpreter.set_tensor(self.input_idx, tensor)
        self.interpreter.invoke()
        probs = self.interpreter.get_tensor(self.output_idx)[0]
        idx = int(np.argmax(probs))
        return self.labels[idx], float(probs[idx]), probs.tolist()
```

A.3 RecommendationEngine Module (Partial)

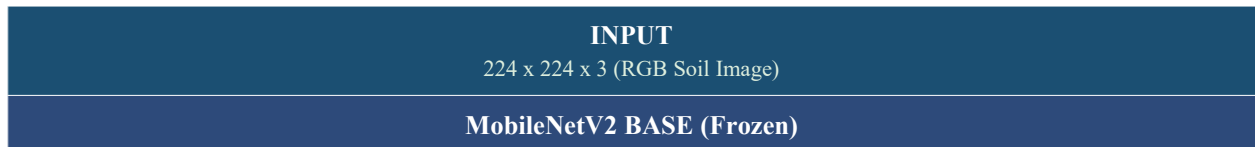
The following extract shows a portion of the RecommendationEngine knowledge base and the recommendation lookup method.

```
RECOMMENDATIONS = {
    'Red': {
        'Maize': {
            'N_kg_ha': 100, 'P_kg_ha': 50, 'K_kg_ha': 30,
            'lime_t_ha': 2.0,
            'products': ['Compound D (300kg/ha)', 'Ammonium Nitrate
(200kg/ha)'],
            'notes': 'Apply lime 4-6 weeks before planting.'
        },
        # ... (20 total soil x crop combinations)
    }
}

def get_recommendation(soil_type: str, crop: str) -> dict:
    if soil_type not in RECOMMENDATIONS:
        raise ValueError(f'Unknown soil type: {soil_type}')
    return RECOMMENDATIONS[soil_type][crop]
```

Appendix B: CNN Model Architecture Diagram

Figure B.1: MobileNetV2 CNN Architecture for Fertile-Map



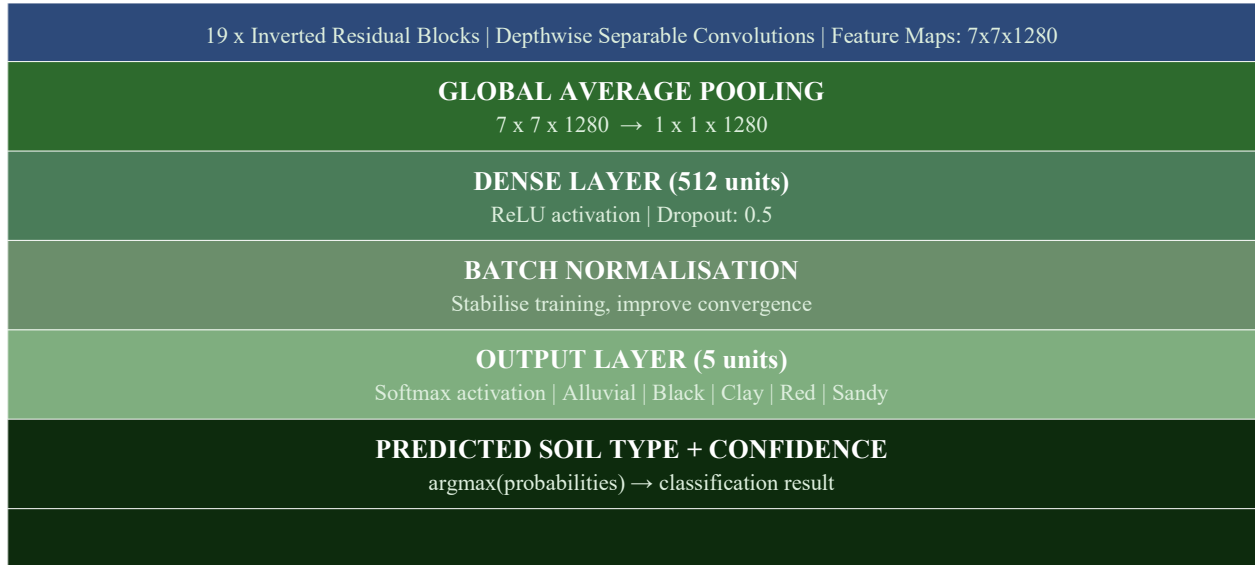


Figure B.1: MobileNetV2 transfer learning architecture adapted for four-class Zimbabwean soil classification.

Appendix C: User Acceptance Testing Survey Instrument

The following questionnaire was administered to all 30 UAT participants following the structured testing session. Responses were collected on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).

Question	Mean Score (n=30)	SD
The application was easy to use	4.3	0.7
I could complete the soil analysis without assistance	4.1	0.9
The soil classification result seemed accurate	3.9	1.1
The fertiliser recommendations were useful and relevant	4.4	0.6
I would use this application on my own farm	4.6	0.5
The application worked well on my smartphone	4.0	0.8
I would recommend this application to other farmers	4.5	0.6

Table C.1: User Acceptance Testing Survey Results (5-point Likert scale, n=30)